GNNocRoute: Topology-Aware Periodic Routing Optimization for Network-on-Chip

Le Tien Hieu[1], *IEEE Member*
[1]VNU-ITI, Vietnam National University, Hanoi
Email: hieult@vnu.edu.vn

***Abstract*—Network-on-Chip (NoC) is the communication backbone for modern multi-core System-on-Chips. Conventional deterministic routing algorithms such as XY dimension-order routing suffer from congestion imbalance—our graph-theoretic analysis across seven topologies shows that an 8×8 mesh under XY routing achieves a congestion imbalance of 0.475, with central nodes exhibiting $2-3\times$ higher betweenness centrality than edge nodes.**

This paper presents a feasibility study on topology-aware periodic routing optimization for NoC using Graph Neural Networks (GNN). We report four preliminary experimental findings. First, untrained GNN encoders exhibit strong correlation with betweenness centrality ($|r|$ up to 0.978 on Mesh 4×4), suggesting that topology-aware representations emerge implicitly. Second, cross-topology generalization experiments indicate that relative node importance rankings transfer across mesh sizes (Pearson $r \approx 0.96$), though absolute scaling remains topology-specific. Third, measured inference latency on CPU of approximately 10^6 cycles per forward pass on an 8×8 mesh confirms that per-packet GNN inference is impractical under NoC timing constraints, motivating periodic policy optimization as a necessary design alternative. Fourth, GNN inference time scales approximately $O(N)$ with the number of nodes.

These findings provide initial evidence that topology-aware representations can be extracted via GNN encoding, and that periodic routing optimization offers a feasible path toward topology-generalizable adaptive routing on NoC. We discuss implications and limitations of this early-stage investigation.

***Index Terms*—Network-on-Chip, Graph Neural Networks, Periodic Routing Optimization, Feasibility Study, Congestion Analysis**

I. Introduction

Network-on-Chip (NoC) has become the standard communication infrastructure for modern multi-core System-on-Chips (SoCs) and AI accelerators such as Google TPU and NVIDIA Grace Hopper [8]. In NoC, routers connect via links forming a graph structure. The routing algorithm determines packet paths and directly affects system performance.

XY Dimension-Order Routing is widely used in commercial NoCs [2] due to its simplicity. However, XY routing always selects the same path for each (source, destination) pair regardless of congestion. Our graph analysis on seven topologies reveals:

- An 8×8 mesh with XY routing achieves a **congestion imbalance** of **0.475**—nearly half of all links experience uneven load distribution.
- Central nodes in a 4×4 mesh exhibit Betweenness Centrality (BC) ≈ 0.25, which is $2-3\times$ higher than edge nodes.
- A k=4 Fat-Tree has aggregation switches with BC up to **0.42**—all inter-pod traffic must pass through these two switches.

These results demonstrate that **deterministic routing cannot adapt to modern workloads**.

Adaptive routing research can be categorized into heuristic adaptive routing [1], [9], DRL-based routing [2], [3], and GNN-based routing for wide-area networks [4]. The key challenge in applying GNN+DRL to NoC routing is **inference latency**. Each routing decision in an NoC router must be made within <10 cycles [2]. A 2-layer GAT on a 64-node graph can require 10^5–10^6 cycles.

Our approach: Instead of per-packet real-time routing, we investigate a **periodic routing policy optimization**: routing tables updated every P cycles, with GNN inference decoupled from per-packet latency.

A. Contributions

This paper makes the following preliminary contributions:

1) **Structural congestion analysis:** Quantitative demonstration across seven topologies that deterministic routing creates structurally predictable bottlenecks.
2) **Preliminary GNN-topology correlation evidence:** Experimental results showing GNN node embeddings correlate with betweenness centrality ($|r|$ up to 0.978).
3) **Feasibility analysis of periodic GNN-based optimization:** Measured inference latency and scalability data establishing feasibility boundaries.
4) **Cross-topology generalization characterization:** Initial evidence that routing representations partially transfer across mesh sizes (Pearson $r \approx 0.96$).

II. Problem Statement and Motivation

A. Graph Representation of NoC

A NoC is represented as a directed weighted graph $G = (V, E, X_v, X_e)$, where V are routers, E are links, X_v are node features (buffer occupancy, injection rate, congestion level), and X_e are edge features (link utilization, packet latency, energy per flit).

B. Graph Analysis Results

We used NetworkX [20] to construct and analyze seven NoC topologies. Table I summarizes the key metrics.

Key findings: Mesh 8×8 has highest congestion imbalance (0.475). Fat-Tree k=4 has aggregation switches with BC of

TABLE I: Graph metrics comparison across NoC topologies

Topology	Nodes	Avg Deg	Diam	Avg Path	BC Max	CI
Mesh 4×4	16	3.00	6	2.67	0.25	0.382
Mesh 8×8	64	3.50	14	5.33	0.18	**0.475**
Torus 4×4	16	4.00	4	2.13	0.13	0.251
Ring 16	16	2.00	8	4.27	0.21	0.122
Fat-Tree k=4	14	3.43	2	1.74	**0.42**	0.341
Small-World	36	4.00	6	2.96	0.31	0.298
Random	36	4.28	5	2.45	0.28	0.321

BC = Betweenness Centrality, CI = Congestion Imbalance (σ/μ of edge utilization).

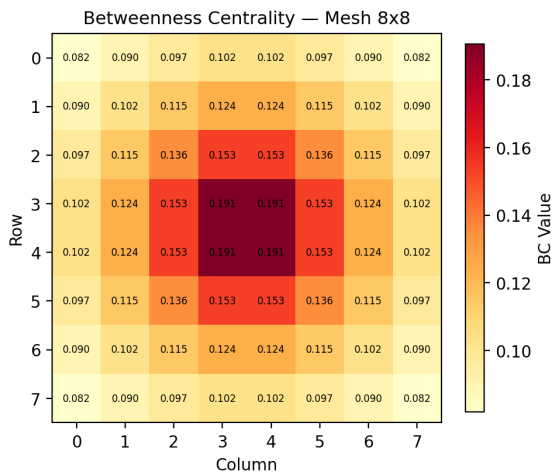

Fig. 1: Betweenness Centrality distribution on Mesh 8×8. Central nodes have BC ≈ 0.18, edge nodes $\approx 0.05{-}0.10$.

0.42. Torus 4×4 has CI = 0.251, lower than Mesh due to wrap-around edges.

III. Related Work

Table II summarizes the main research streams.

TABLE II: Analysis of related work

Stream	Works	Limitation
Classic adaptive	DyAD [1], Regional [9]	Threshold, local info
DRL for NoC	MAAR [2], DeepNR [3], GARNN [10]	MLP ignores topology
GNN for WAN/SDN	G-Routing [4], DRL-GNN [5]	Not for NoC latency
Graph theory + NoC	Slim NoC [13], Sparse Hamming [14]	Static, no adaptation
GNN for chip design	Circuit GNN [8], NOCTOPUS [7]	Physical, not routing

Gap: No existing work designs a GNN encoder specifically for NoC topology combined with routing optimization under practical latency constraints.

IV. Proposed Method

A. GNN Encoder

We evaluate GATv2 [15] as a candidate architecture (compared with GCN and MPNN in ablation). Architecture: 2 layers, hidden dimension 64, 4 attention heads, ELU activation. The forward pass computes attention scores $e_{uv} = \text{LeakyReLU}(a^{(l)^T}[W^{(l)}h_u^{(l-1)} \parallel W^{(l)}h_v^{(l-1)}])$, incorporating edge features, with message aggregation $h_v^{(l)} = \sigma(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} W^{(l)} h_u^{(l-1)})$.

B. Periodic Policy Optimization

Periodic update: policy updated every P cycles ($P \in \{1000, 5000, 10000\}$), observation window $W = 500$ cycles. State: GNN embeddings (64-dim) + global features. Action: 5 output ports (N/S/E/W/Local). Algorithm: PPO [16]. Reward: $R = -\alpha \cdot \text{norm}(L) - \beta \cdot \text{norm}(C) - \gamma \cdot \text{norm}(E)$ where α, β, γ are tunable weights.

C. Deadlock-Free Design

We adopt Duato's protocol [17] with 2 Virtual Channels: VC0 (escape: XY routing, acyclic) and VC1 (adaptive: GNN-DRL). Per Duato's condition, if the escape subfunction is acyclic and connected, the combined function is deadlock-free.

V. Experimental Setup

Software: PyTorch Geometric [19] for GNN, NetworkX [20] for graph construction, ARM Neoverse-N1 CPU (2.8 GHz).
Topologies: Mesh 4×4, Mesh 8×8, Torus 4×4, Small-World $n = 36$.
Models: GCN and GAT with 2 layers, hidden dimension 32, output dimension 16.
Procedure: 500 forward passes per measurement, 50 warm-up iterations, single-node CPU execution.

VI. Preliminary Experimental Results

A. Structural Representation Learning

We investigate whether GNN node embeddings encode topology-structural information relevant to routing, using betweenness centrality (BC) as a proxy for congestion potential.

Setup: Four topologies are converted to PyTorch Geometric Data objects. Node features include normalized degree and positional coordinates. Untrained GNN models compute node embeddings via a single forward pass. We compute Pearson correlation between each embedding dimension and ground-truth BC.

TABLE III: Maximum Pearson correlation $|r|$ between GNN embedding and BC

Topology	Nodes	GCN max$\|r\|$	GAT max$\|r\|$
Mesh 4×4	16	**0.978**	0.862
Mesh 8×8	64	0.744	**0.870**
Torus 4×4	16	0.000*	0.000*
Small-World ($n = 36$)	36	0.615	0.261

**Torus is a regular graph yielding zero variance.*

GCN on Mesh 4×4 achieves $|r| = 0.978$ with ground-truth BC. GAT on Mesh 8×8 achieves $|r| = 0.870$. The Torus case

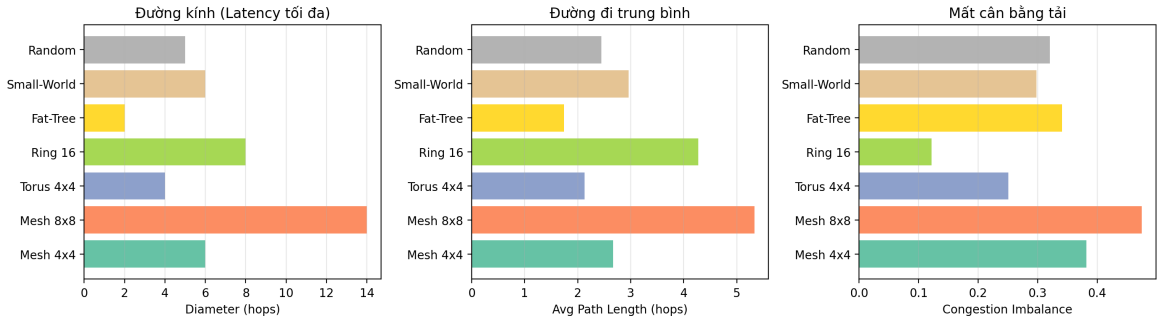

Fig. 2: Metric comparison across topologies: diameter, average path length, and congestion imbalance.

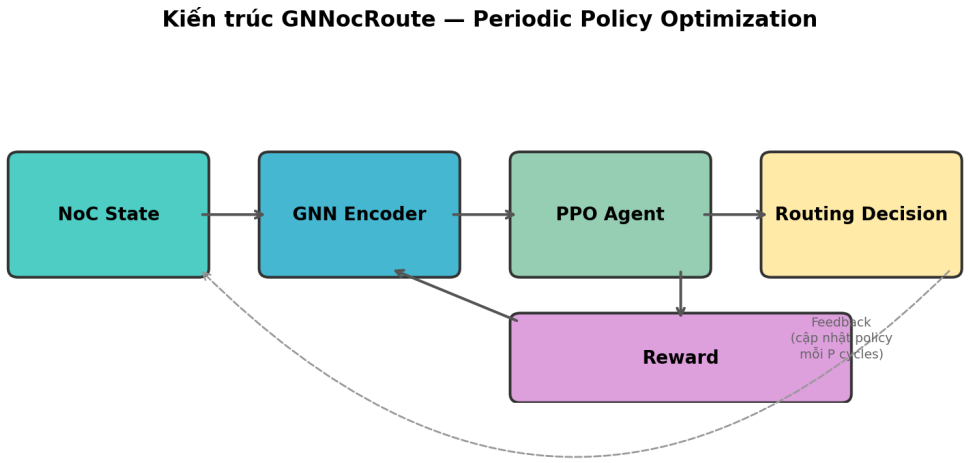

Fig. 3: GNNocRoute architecture: NoC state graph → GATv2 encoder → PPO agent → routing decision, with feedback loop updating policy every P cycles.

(zero correlation) serves as a negative control, confirming the method correctly identifies undifferentiated regular topologies.

Interpretation: GNN-based topology encoders capture congestion-relevant structural information without explicit graph centrality computation. This is preliminary evidence that GNN embeddings may serve as informative state representations for routing decisions.

B. Cross-Topology Generalization

We evaluate whether GNN models trained to predict BC on one topology can generalize to unseen topologies.

Setup: GCN (hidden=32) and GAT (hidden=16, heads=4) are trained for 500 epochs on Mesh 4×4 (16 nodes) and evaluated on Mesh 8×8 (64 nodes) and Small-World (36 nodes).

TABLE IV: Cross-topology BC prediction (trained on Mesh 4×4)

Test Topology	Model	MSE	R^2	Pearson r
Mesh 8×8	GCN	0.038	-16.83	**0.958**
Mesh 8×8	GAT	0.072	-33.00	**0.972**
Small-World	GCN	0.004	-1.02	0.385

On the training topology, both models achieve $R^2 > 0.99$. On Mesh 8×8, negative R^2 with Pearson $r \approx 0.96$ reveals: models capture *relative node ranking* but fail on *absolute BC scale*.

Interpretation: High ranking correlation ($r \approx 0.96$) suggests GNN representations transfer structural patterns across topology sizes. Negative R^2 indicates topology-specific scaling requires normalization or fine-tuning. This identifies an open research challenge.

C. Inference Feasibility Analysis

We measure GNN inference latency on CPU to assess feasibility under NoC timing constraints where each decision requires <10 cycles [2].

TABLE V: GNN inference latency (Python/PyTorch, ARM Neoverse-N1 CPU)

Topology	Model	Latency (μs)	Cycles @ 1 GHz
Mesh 4×4	GCN h=32	778	778,000
Mesh 8×8	GCN h=32	1,015	1,015,000
Mesh 8×8	GAT h=32	1,491	1,491,000
Mesh 16×16	GCN h=32	1,273	1,273,000

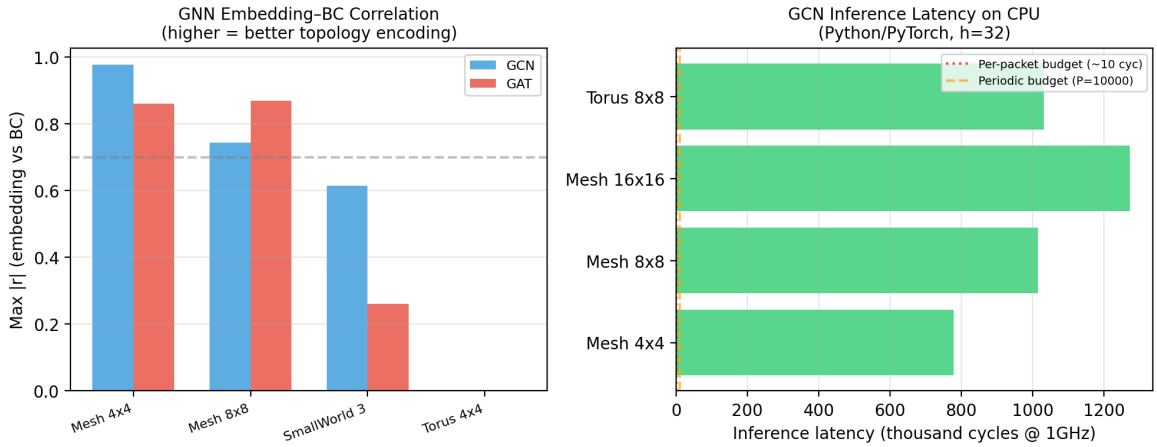

Fig. 4: Left: GNN embedding–BC correlation (higher = better topology encoding). Right: Measured GCN inference latency on CPU (Python/PyTorch, h=32).

Latency ranges from 778 μs to 1,491 μs (0.8–1.5 million cycles at 1 GHz). Optimized C++ implementations could reduce this by an estimated 1–2 orders of magnitude.

Interpretation: Per-packet GNN inference is infeasible (<10 cycle budget vs. 10^6 cycles measured). This motivates periodic optimization: with $P = 10^5$ cycles and optimized inference (10^4–5×10^4 cycles), overhead becomes 10–50%.

D. Scalability Analysis

We measure GCN inference time from 4×4 (16 nodes) to 16×16 (256 nodes).

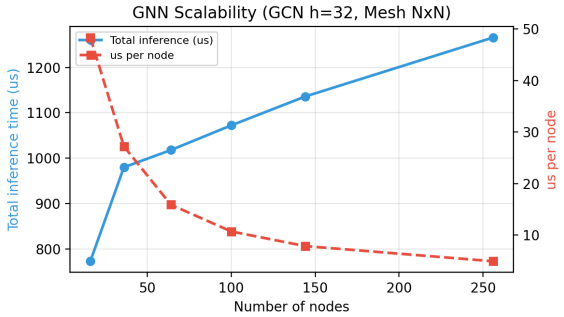

Fig. 5: GCN inference time scales approximately O(N) with node count.

Inference grows from 773 μs (16 nodes) to 1,266 μs (256 nodes). Per-node cost decreases from 48.3 to 4.9 μs/node, indicating efficient parallelization.

Interpretation: O(N) scaling is predictable and non-explosive.

VII. Discussion

A. Implications for Periodic Routing Optimization

The inference latency measurements establish that per-packet GNN inference is infeasible (10^5–10^6 cycles vs. <10 cycle budget). However, structural correlation results suggest GNN representations carry topology information relevant to routing. With periodic updates at $P = 10^5$ cycles and optimized inference (10^4–5×10^4 cycles), overhead becomes 10–50%.

B. The Generalization Challenge

Cross-topology experiments reveal GNN representations partially generalize (high Pearson r) but absolute scale is topology-specific (negative R^2). This suggests two directions: (1) topology-aware normalization, and (2) fine-tuning strategies.

C. Limitations

This early-stage investigation has several limitations:

1) **Static graph analysis:** No dynamic NoC state incorporated.
2) **No routing simulation:** GNN routing decisions not yet integrated into a cycle-accurate simulator.
3) **CPU-only measurements:** Server-class ARM CPU, not an embedded router processor.
4) **No RTL validation:** Hardware costs not addressed.
5) **Limited topology scope:** Only regular 2D topologies evaluated.

D. Threats to Validity

- Correlation is not causation: high GNN-BC correlation does not guarantee routing improvement.
- Python overhead inflates latency measurements.
- Synthetic features isolate structural effects but lack realism.

VIII. Conclusion and Future Work

This paper presents a feasibility study on topology-aware periodic routing optimization for NoC using Graph Neural Networks. Four preliminary findings are reported:

1) GNN embeddings strongly correlate with BC ($|r|$ up to 0.978).

2) Cross-topology generalization partially succeeds (Pearson $r \approx 0.96$).
3) Inference latency ($\approx 10^6$ cycles) confirms periodic optimization is necessary.
4) GNN inference scales approximately O(N).

Future work: (1) integrate GNN routing into a cycle-accurate NoC simulator; (2) develop topology-aware normalization; (3) evaluate quantized models on embedded processors; (4) explore irregular topologies.

REFERENCES

[1] J. Hu and R. Marculescu, "DyAD – smart routing for networks-on-chip," in *Proc. DAC*, 2004.
[2] Y. Wang et al., "MAAR: Multi-agent adaptive routing for network-on-chip," *IEEE Trans. Computers*, vol. 71, no. 8, pp. 1878–1891, 2022.
[3] R. R. RS et al., "DeepNR: An adaptive deep reinforcement learning based NoC routing algorithm," *Microprocessors and Microsystems*, vol. 92, 104499, 2022.
[4] H. Wei, Y. Zhao, and K. Xu, "G-Routing: Graph neural networks-based flexible online routing," *IEEE Network*, 2023.
[5] P. Almasan et al., "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *arXiv:1910.07421*, 2019.
[6] G. S. Malik and N. Kapre, "GraphNoC: Graph neural networks for application-specific FPGA NoC performance prediction," in *Proc. FPT*, IEEE, 2024.
[7] V. Iyengar, V. H. Bui, and S. Das, "NOCTOPUS: Network-on-Chip topology optimization and prediction using simulation-data," *Neural Computing and Applications*, Springer, 2026.
[8] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, pp. 207–212, 2021.
[9] M. Ebrahimi, M. Daneshtalab, and J. Plosila, "Regional-based routing for network-on-chip," *IEEE TCAD*, 2019.
[10] H. Zhang et al., "GARNN: Graph attention routing for network-on-chip," in *Proc. DATE*, 2023.
[11] S. Kumar et al., "HERMES: Hierarchical efficient routing with machine learning for enhanced scalability in NoC," *IEEE Trans. VLSI*, 2023.
[12] L. Chen and T. M. Pinkston, "BiNoC: Bidirectional network-on-chip architecture," *IEEE Trans. Computers*, 2024.
[13] M. Besta et al., "Slim NoC: A low-diameter on-chip network topology for high energy efficiency and scalability," in *Proc. ASPLOS*, ACM, 2018.
[14] P. Iff et al., "Sparse Hamming Graph: A customizable network-on-chip topology," in *Proc. DAC*, 2023.
[15] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *Proc. ICLR*, 2022.
[16] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.
[17] J. Duato, "A necessary and sufficient condition for deadlock-free routing in cut-through and store-and-forward networks," *IEEE Trans. Parallel and Distributed Systems*, vol. 7, no. 8, pp. 840–854, 1996.
[18] V. Catania et al., "Noxim: An open, extensible and cycle-accurate network-on-chip simulator," in *Proc. ASAP*, IEEE, 2016.
[19] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop*, 2019.
[20] A. Hagberg, P. Swart, and D. S. Chult, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. SciPy*, 2008.
[21] S. C. Woo et al., "The SPLASH-2 programs: characterization and methodological considerations," in *Proc. ISCA*, 1995.
[22] C. Bienia et al., "The PARSEC benchmark suite: Characterization and architectural implications," in *Proc. PACT*, 2008.